

Channel Pruning Framework — Fix & Consolidation Log

Scope agreed with the repo owner: focus on the **channel pruning** pipelines (distance/similarity and saliency criteria), consolidate each pipeline's duplicated "base" and "vgg_" script into one canonical file per strategy, and fix everything for correctness even where it changes numeric results from past runs. Kernel pruning files (`kernel_pruning_*.py` , `vgg_kernel_pruning_*.py`) were left untouched — out of scope for this pass.

Validation performed: `python3 -m py_compile` on every changed file (passes). No PyTorch install exists on this machine, so no runtime/GPU execution of the pruning loop has happened yet — that still needs to be run on the actual training machine before trusting new pruning results end-to-end.

1. `train_model.py`

Bug — `evaluate()` only scored the last batch. The validation loop built a list literal *inside* the loop body:

```
for batch_X, batch_y in data_loader:
    outputs = [compute_batch_loss_acc(model, batch_X, batch_y)]
return accumulate_batch_loss_acc(outputs)
```

Each iteration replaced `outputs` instead of appending to it, so after the loop `outputs` held only the last batch's result. Every reported `val_loss / val_acc` was computed from a single batch, not the full validation set.

Fix: build the full list with a comprehension, then accumulate once:

```
outputs = [compute_batch_loss_acc(model, batch_X, batch_y) for batch_X, batch_y in data_loader]
return accumulate_batch_loss_acc(outputs)
```

Bug — validation ran every batch, not once per epoch. `fit_one_cycle()` called `evaluate()` and wrote `result['train_loss'] / lrs` *inside* the per-batch training loop (indentation put it one level too deep), so validation reran on every single training batch — wasteful, and the epoch summary ended up describing only the final batch's state rather than the epoch as a whole.

Fix: dedented the evaluate/record block to run once per epoch, after the batch loop finishes.

Bug — L1 regularization was unconditional and covered frozen parameters.

```
l1_crit = nn.L1Loss()
reg_loss = 0
for param in model.parameters():
    reg_loss += l1_crit(param, target=torch.zeros_like(param))
loss += L1 * reg_loss
```

This always added L1 penalty terms — including for parameters with `requires_grad=False` (e.g. frozen conv layers) — and ran even when the caller passed `L1=0`, doing pointless extra work every batch.

Fix: gate on `if L1:`, and only regularize trainable parameters:

```
if L1:
    ll_crit = nn.L1Loss()
    reg_loss = sum(ll_crit(param, target=torch.zeros_like(param))
                   for param in model.parameters() if param.requires_grad)
    loss = loss + L1 * reg_loss
```

Impact: every previously-recorded validation accuracy/loss number from this codebase reflects one random batch, not the real validation set. Fine tuning was also up to `num_batches` times slower than necessary per epoch. Numbers from prior runs are not comparable to numbers produced after this fix.

2. `load_model.py`

Bug — `freeze()` never unfroze anything.

```
def freeze(model, model_name):
    count = 0
    if model_name == 'vgg16':
        for param in model.parameters():
            if count == 30:
                param.requires_grad = True
            else:
                param.requires_grad = False
        count = count + 1 # <- outside the for-loop: dead code, count never advances
    if model_name == 'vgg13': ... # no count increment at all
    if model_name == 'vgg11': ... # no count increment at all
```

`count` was incremented once, after the loop had already finished — a no-op — so `count == 30` was never true inside the loop, and **every** parameter in the model ended up with `requires_grad = False`. The intended "freeze everything except the last classifier layer" behavior never actually happened for any VGG variant.

Separately, the hardcoded parameter-position numbers (30 / 24 / 20) were derived by counting parameters of a *non-BatchNorm* VGG. Every caller in this codebase actually instantiates BatchNorm VGG variants (`batch_norm=True`), whose extra per-conv BN weight/bias parameters shift every subsequent index — so even with the counter bug fixed, those magic numbers would still point at the wrong parameter.

Fix: target the classifier's last layer directly by structure instead of by a hardcoded flat parameter index, which is correct for any VGG depth or BatchNorm variant:

```
def freeze(model, model_name=None):
    for param in model.parameters():
        param.requires_grad = False
    for param in model.classifier[-1].parameters():
        param.requires_grad = True
```

`model_name` is kept as an accepted-but-unused parameter so existing call sites don't need to change signature-wise (though all call sites were in fact simplified to drop the argument — see below).

Impact: this is the mechanism meant to keep the pretrained convolutional backbone frozen while fine-tuning only the final classifier during transfer learning. Because it silently froze *everything*, transfer-learning runs that relied on it were training zero parameters in the backbone and, if the classifier head also matched by accident, potentially training nothing at all.

3. `initialize_pruning.py`

Bug — trailing commas turned VGG feature lists into 1-tuples.

```
vgg16 = [64, 64, "M", 128, 128, "M", 256, 256, 256, "M", 512, 512, 512, "M", 512, 512, 512, "M"],
```

The trailing comma after the closing `]` makes the right-hand side a 1-element tuple *containing* the list, not the list itself — i.e. `vgg16 == ([64, ...],)`. Anything consuming `get_feature_list('vgg16')` (used to rebuild a VGG's `.features` stack from a feature list after pruning) would receive a tuple instead of a list and either crash or behave unpredictably depending on how it was indexed.

Fix: removed the trailing commas from `vgg11`, `vgg13`, `vgg16`, `vgg19`.

4. `load_dataset.py`

Bug — `CenterCrop` used as a bare class reference instead of a transform instance.

```
transforms.Resize(256),
transforms.CenterCrop,      # <- missing (224) call
transforms.ToTensor(),
```

`transforms.CenterCrop` (no parens/args) is the class itself, not a configured transform — `Compose` would call it on a tensor with no crop size, which raises at transform-application time.

Fix: `transforms.CenterCrop(224)`.

5. facilitate_pruning.py

This file holds the two channel-importance scoring functions and the weight-copy routine used to shrink a model after pruning. All three had correctness bugs; all three were rewritten.

5a. `compute_saliency_score_channel` — dead/ $O(n^4)$ manual max-finding, replaced

Before: ~35 lines of manual nested loops recomputing per-channel norms with hand-rolled "top-3 absolute value" tracking that was never actually used in the returned score (the function computed `channel_norm[i] += max1+max2+max3` using unrelated bookkeeping variables, then separately did an $O(n^2)$ selection-sort-style manual argmin loop to pick the `prune_amount` smallest). The two computed norms (`channel_norm` vs `channel_norm_temp`) diverged in value but were only synchronized by index, not content, once `channel_norm[i]` was overwritten inside the max-tracking loop.

After: standard Lp-norm channel-magnitude ranking (the actual intended criterion — cf. Li et al., "Pruning Filters for Efficient ConvNets"):

```
def compute_saliency_score_channel(tensor_t, n=1, prune_amount=1):
    out_channels = tensor_t.shape[0]
    prune_amount = max(0, min(prune_amount, out_channels - 1))
    channel_norm = torch.norm(tensor_t.reshape(out_channels, -1), p=n, dim=1)
    order = torch.argsort(channel_norm)
    return order[:prune_amount].tolist()
```

Also changed the **return type**: previously returned `[[idx, norm_value], ...]` pairs; now returns a plain list of channel indices, weakest first. This matches what every caller actually needs (indices to zero out) and removes an entire layer of `[k1][k2][0] / [1]` index-unpacking gymnastics that existed downstream in the pruning-mask classes (see §7).

5b. `compute_distance_score_channel` — indexing bug made every "distance" identical

Before:

```
for i in range(size[0]):
    scale_tensor = tensor_t[i]/torch.norm(tensor_t[[i]])
```

`scale_tensor` is reassigned (not indexed/appended) on every loop iteration, so after the loop it holds only channel `size[0]-1`'s normalized value. Every subsequent pairwise "distance" comparison, `torch.norm(scale_tensor[i1] - scale_tensor[i2])`, then indexes *that same single normalized channel* for both `i1` and `i2` — the result is always `torch.norm(scale_tensor[i1] - scale_tensor[i2])` computed on the same underlying tensor sliced two different ways, not a real distance between channel `i1` and channel `i2` of the original weight tensor. The "redundancy" pruning criterion this feeds was therefore not measuring inter-channel redundancy at all.

After: normalize every channel, then compute real pairwise distances and return the channels with the smallest nearest-neighbor distance (i.e. the most redundant ones):

```
def compute_distance_score_channel(tensor_t, n=1, prune_amount=1):
    out_channels = tensor_t.shape[0]
    prune_amount = max(0, min(prune_amount, out_channels - 1))
    flat = tensor_t.reshape(out_channels, -1)
    norms = torch.norm(flat, p=2, dim=1, keepdim=True).clamp_min(1e-12)
    normalized = flat / norms
    dist = torch.cdist(normalized, normalized, p=n)
    dist.fill_diagonal_(float('inf'))
    nearest_dist, _ = torch.min(dist, dim=1)
    order = torch.argsort(nearest_dist)
    return order[:prune_amount].tolist()
```

Same return-type simplification as §5a: a plain list of channel indices. `clamp_min(1e-12)` guards a zeroed-out channel (norm 0) from producing a `NaN` via division by zero — relevant because this function now also runs in later pruning rounds against already-pruned (partially zeroed) models.

5c. `deep_model_copy_channelwise` — three separate bugs, all silent or fatal

Before:

```
def deep_model_copy_channelwise(source_model, destination_model, feature_list):
    for l in range(len(source_model.features)):
        if str(source_model.features[l]).find('Conv') != -1:
            size_org = source_model.features[l]._parameters['weight'].shape
            out_ch_new = 0
            for out_ch_old in range(size_org[0]):
                if torch.norm(source_model.features[l]._parameters['weight'][out_ch_old] != 0):
                    t = source_model.features[l]._parameters['weight'][out_ch_old]
                    destination_model.features[l]._parameters['weight'][out_ch_new] = t
                    out_ch_old += 1
```

1. **`out_ch_new` is never incremented** — every surviving channel gets written into destination slot 0, clobbering all but the last surviving channel of every conv layer.
2. **`out_ch_old += 1` is dead code** — `out_ch_old` is the `range()` loop variable; reassigning it inside the loop body has no effect on iteration.
3. **`torch.norm(x != 0)` is not "is this channel non-zero."** `x != 0` is already a boolean tensor; `torch.norm` of a boolean tensor is always truthy for any tensor with at least one nonzero element — in practice this condition is `True` for essentially every channel regardless of whether pruning actually zeroed it, because it's testing "does this channel have at least one non-zero *comparison result*" rather than "is this channel entirely zero."
4. **Input-channel dimension was never re-sliced.** Even ignoring 1–3: pruning shrinks layer L 's output-channel count. Layer $L+1$'s weight tensor has shape `[out, in, kh, kw]` where `in` must equal layer L 's surviving output count. The old code copied each layer's full input-channel slice unmodified, so for any conv layer beyond the first, the copied slice's input dimension (still source-model-sized) would not match `destination_model`'s already-shrunk weight tensor shape — a shape mismatch that raises at copy time, or (in dry runs without pruning) is masked because with a size-preserving no-op it happens to still fit.
5. **BatchNorm parameters were never copied at all** — for the `batch_norm=True` models this codebase actually uses, that means every fine-tuning round after the first started from freshly re-initialized BatchNorm running statistics instead of the trained ones, discarding normalization statistics learned in the previous round.

After:

```
def deep_model_copy_channelwise(source_model, destination_model, feature_list):
    prev_surviving_in = None # channel indices kept by the previous conv layer
    with torch.no_grad():
        for l in range(len(source_model.features)):
            layer_str = str(source_model.features[l])
            if layer_str.find('Conv') != -1:
                src_weight = source_model.features[l]._parameters['weight']
                dst_weight = destination_model.features[l]._parameters['weight']
                src_bias = source_model.features[l]._parameters.get('bias')
                dst_bias = destination_model.features[l]._parameters.get('bias')

                out_org = src_weight.shape[0]
                surviving_out = [o for o in range(out_org) if torch.norm(src_weight[o]) != 0]

                for new_o, old_o in enumerate(surviving_out):
                    src_slice = src_weight[old_o]
                    if prev_surviving_in is not None:
                        src_slice = src_slice[prev_surviving_in]
                    dst_weight[new_o] = src_slice
                    if src_bias is not None and dst_bias is not None:
                        dst_bias[new_o] = src_bias[old_o]

                prev_surviving_in = surviving_out
            elif layer_str.find('BatchNorm') != -1 and prev_surviving_in is not None:
                src_bn = source_model.features[l]
                dst_bn = destination_model.features[l]
                for new_o, old_o in enumerate(prev_surviving_in):
                    dst_bn.weight[new_o] = src_bn.weight[old_o]
                    dst_bn.bias[new_o] = src_bn.bias[old_o]
                    dst_bn.running_mean[new_o] = src_bn.running_mean[old_o]
                    dst_bn.running_var[new_o] = src_bn.running_var[old_o]
```

- `torch.norm(src_weight[o]) != 0` correctly tests "is this whole channel's weight tensor all-zero" (real L2 norm of the channel, compared to 0 — not a boolean-of-a-comparison).
- `enumerate(surviving_out)` gives a correctly-advancing destination index.
- `prev_surviving_in` carries forward the previous conv layer's surviving output-channel indices, and slices the *current* layer's input dimension with them before copying — fixing the shape-mismatch bug for any conv layer beyond the first.
- BatchNorm weight/bias/running_mean/running_var are now copied using the same surviving-channel indices, using the immediately preceding conv layer's surviving set (a BN layer always immediately follows its conv layer in these VGG feature stacks, so `prev_surviving_in` at that point is exactly the right set).
- Wrapped in `torch.no_grad()` since this is a manual in-place weight copy, not an autograd-tracked operation.

Impact: this function is the core mechanism that shrinks a model after each pruning round. In its previous state, it either crashed on any multi-conv-layer VGG (once the shape mismatch was hit) or, in code paths that happened to avoid the crash, silently produced a destination model whose weights did not correspond to the intended surviving channels and whose BatchNorm statistics were freshly reinitialized every round.

6. config.ini

Added: a [Pruning] section with `max_pruning_ratio`, `prune_epochs`, `fine_tune_epochs`, `batch_size`, `image_size`, `max_lr`, `weight_decay`, `l1_lambda`, `grad_clip`. All of these were previously hardcoded directly in each driver script (`max_pr=0.1`, `epochs=8`, `max_lr=0.001`, etc.) — moving them to config means a run's hyperparameters are visible/versioned in one place instead of buried in three near-duplicate scripts, and the two consolidated channel-pruning scripts now read identical values from the same source instead of each hardcoding their own (previously-consistent-by-luck) copies.

Changed: `program_name` from `vgg_net_kernel_pruning` to `vgg_net_channel_pruning` (this config is used by the channel-pruning scripts; the old value would have written channel-pruning outputs into a kernel-pruning-labeled log/model directory). Added `num_classes = 6` (was hardcoded as `6` in every driver script). Removed the unused `model_dir` / `logDir` keys, which had been superseded by `dir_home_path`-based path construction that the actual scripts use (`dir_home_path` already existed under [Dataset]; the two removed keys were dead config that no script read).

7. channel_pruning_distance.py / channel_pruning_saliency.py (consolidation)

Per your direction, each of these pairs:

- `channel_pruning_distance.py` + `vgg_channel_pruning_dist.py`
- `channel_pruning_saliency.py` + `vgg_channel_pruning_saliency.py`

was consolidated into a single canonical script (the `channel_pruning_*.py` name was kept; the `vgg_channel_pruning_*.py` files were deleted — see §9). Beyond folding in the surviving useful logic from the `vgg_` variants, both scripts had the same set of bugs, fixed identically in both:

Bug — config values read, then silently overwritten with hardcoded values a few lines later. e.g. in the saliency script:

```
if config.get('Model','loadModel') == 'True':
    loadModel = True
else:
    loadModel = False
...
loadModel = False          # <- clobbers whatever config said
is_transfer_learning = False # <- clobbers whatever config said
```

So `config.ini`'s `loadModel` / `is_transfer_learning` settings had no effect — the script always ran with `loadModel=False`, `is_transfer_learning=False` regardless of what was configured.

Fix: read each value from config exactly once via `config.getboolean(...)` and never reassign it afterward.

Bug — `dl.set_batch_size = 16` assigns over the function instead of calling it.

```
dl.set_batch_size = 16
```

This rebinds the module attribute `set_batch_size` (previously a function) to the integer `16`, rather than calling `dl.set_batch_size(16)`. Any later code in `load_dataset.py` that calls `dl.set_batch_size(...)` as a function would now raise `TypeError: 'int' object is not callable`; more subtly, the batch size the data loader actually uses is whatever `load_dataset.py`'s internal default was, not 16, since the "setter" never ran.

Fix: `dl.set_batch_size(batch_size)` (calling the function, with the value now sourced from `config.ini`).

Bug — `dataset_dir / selected_dataset_dir` path construction was inconsistent between the two scripts and, in the saliency script, `selected_dataset_arg=''` was passed to `dl.data_loader(...)` (an empty string instead of the actual selected dataset folder name), which would build a dataset path missing its dataset-name path segment.

Fix: both scripts now build `dataset_dir` identically from `dir_home_path + 'Dataset/'` and pass the real `selected_dataset_dir` through.

Bug — `layer_number` global used by the pruning-mask `compute_mask()` was never updated to the layer actually being pruned. Both original mask classes (`ChannelPruningMethodSimilarities` in the distance script, `ChannelPruningMethodSaliency` in the saliency script) computed `i = layer_number - layer_base` to index into the current block's candidate list, but nothing in the surrounding loop ever set `layer_number` to anything other than its initial value of `0`. Every layer after the first in each block therefore got masked using **layer 0's** candidate list, regardless of which layer was actually being pruned.

Separately, **saliency's mask class had the actual masking line commented out and an unconditional `break`**:

```
for j in range(len(new_list[i])):
    k = new_list[i][j][0]
    print("value of k is:", k)
    #mask[k] = 0      # <- the only line that would zero a channel, disabled
    break            # <- always exits after the first candidate regardless
```

This means saliency-based channel pruning was doing **nothing** — every "pruning" round ran fine-tuning on the full, unpruned dense model while logging/saving as if pruning had occurred.

Fix: replaced the global-`layer_number`-indexed mask classes in both scripts with a single, simpler pruning-mask mechanism driven by an explicit list of channel indices per call, with no global layer-tracking state to get out of sync:

```
_channels_to_prune = []

class ChannelPruningMask(prune.BasePruningMethod):
    PRUNING_TYPE = 'unstructured'

    def compute_mask(self, t, default_mask):
        mask = default_mask.clone()
        for ch in _channels_to_prune:
            mask[ch] = 0
        return mask

    def prune_channels(conv_module, name, channel_indices):
        global _channels_to_prune
        _channels_to_prune = channel_indices
        ChannelPruningMask.apply(conv_module, name)
        return conv_module
```


Each conv layer's candidate indices (now plain index lists, per the §5a/§5b return-type change) are computed and applied to that exact layer in the same loop iteration — there's no separate global counter that can drift out of sync with which layer is actually being processed.

Bug — `prune_count` computed from `module` before `module` was populated, in the original distance script:

```
def initialize_lists_for_pruning():
    global block_list, feature_list, conv_layer_index, prune_count, module
    block_list = ip.create_block_list(new_model)
    feature_list = ip.create_feature_list(new_model)
    conv_layer_index = ip.find_conv_index(new_model)
    prune_count = ip.get_prune_count(module=module, blocks=block_list, max_pr=.1) # module is still []
    module = ip.make_list_conv_param(new_model) # populated *after* being used above
```

`get_prune_count` was called with `module` still at its initial empty-list value, so `prune_count` was computed against zero conv layers (empty result), then `module` was populated afterward — too late to affect the already-computed `prune_count`. (The saliency script/sibling scripts had this in the correct order already, so this looked like a regression rather than an intentional design choice.)

Fix: the consolidated main loop computes `conv_modules`, then `prune_count` from it, in the correct order, once per pruning round (see below) rather than once globally at import time.

Structural change — per-round recomputation instead of one-time global init. The original scripts called `initialize_lists_for_pruning()` once at import time and reused the same `block_list` / `prune_count` / `feature_list` values for every one of the 6 pruning rounds — even though each round changes the model's actual channel counts. The consolidated main loop (`iterative_channel_pruning_distance_block_wise` / `_saliency_block_wise`) now recomputes `block_list`, `conv_modules`, `prune_count`, and `feature_list` from the **current** model at the top of every round:

```
for e in range(prune_rounds):
    block_list = ip.create_block_list(model)
    conv_modules = ip.make_list_conv_param(model)
    prune_count = ip.get_prune_count(module=conv_modules, blocks=block_list, max_pr=max_pruning_ratio)
    feature_list = ip.create_feature_list(model)
    ...
    model = temp_model # next round prunes the model this round just produced
```

This also fixes a related bug: the original loop always pruned from the *same* `new_model` / `prune_module` captured at import time — i.e. round 2 computed pruning candidates from round 1's *original* (pre-pruning) weights rather than the fine-tuned, already-shrunk model round 1 actually produced, even though `temp_model` (round 1's output) was correctly used for *fine-tuning*. The candidate-selection step and the fine-tuning step were operating on two different models. Now both operate on the same progressively-shrinking `model` / `temp_model` each round.

Removed: hardcoded `epochs=8`, `max_lr=0.001`, `weight_decay=0.01`, `L1=0.01`, `grad_clip=0.1`, `max_pr=.1` and the hardcoded `model_name='vgg16'` / `vgg16_IntelIc_Prune_{e}_b_train` save-path segments — all now sourced from `config.ini` (see §6) or built from `program_name` / `selected_dataset_dir` already read from config.

Fixed a broken save path that had a literal `/selected/dataset_dir/` segment (looks like a variable interpolation that was never turned into an f-string):

```
save_path = f'{model_dir}{program_name}/selected/dataset_dir/vgg16_IntelIc_Prune_{e}_b_train'
```

→

```
save_path = f'{model_dir}/vgg16_IntelIc_Prune_{e}_b_train'
```

(`model_dir` already includes `program_name / selected_dataset_dir` via the config-driven path construction in §6/§7.)

Fixed dead evaluation code: both original scripts computed `train_accuracy = 0.0 / test_accuracy = 0.0` as literal constants and logged those constants — evaluation was never actually run after fine-tuning, so every per-round logged accuracy was `0.0` regardless of the model's real accuracy. Now calls `tm.evaluate(...)` on both the train and test loaders and logs the real `val_acc`.

vgg16 → vgg16bn: both scripts now load/rebuild `vgg16bn` (BatchNorm) instead of plain `vgg16`, matching what `lm.create_vgg_from_feature_list(..., batch_norm=True)` rebuilds every round — the source and rebuilt models must have the same `.features` layer indexing for `deep_model_copy_channelwise` (§5c) to copy into the correct layers. Loading plain `vgg16` while every subsequent round rebuilds a BatchNorm model was a pre-existing architecture mismatch bug, not something introduced by this pass. All `lm.freeze(...)` call sites were simplified to drop the now-unused `model_name` argument (see §2).

8. Deep copy call was previously reading from the wrong "current" model

(Folded into §7's "per-round recomputation" note, called out separately here since it's a distinct bug from the stale `prune_count / block_list` issue.) Both original scripts always passed the import-time `new_model` into `deep_model_copy_channelwise(new_model, temp_model, ...)` on every round, instead of the previous round's fine-tuned output. Fixed by threading `model` through the loop and reassigning `model = temp_model` at the end of each round.

9. Deleted files

- `vgg_channel_pruning_dist.py` — unfinished duplicate of `channel_pruning_distance.py`; useful pieces folded in during consolidation (§7), remainder was dead/stub code with no iterative-fine-tuning driver wired up.
- `vgg_channel_pruning_saliency.py` — unfinished duplicate of `channel_pruning_saliency.py`; same treatment.

Both deletions were explicitly confirmed with the repo owner before removal, per the original consolidation plan ("remove the redundant one after you confirm").

Kernel pruning's own vgg/base duplication (`kernel_pruning_saliency.py`, `kernel_pruning_similarities.py`, `vgg_kernel_pruning_dist.py`, `vgg_kernel_pruning_saliency.py`) was **not** touched — out of scope for this pass — and still has known issues (`kernel_pruning_similarities.py`'s `temp_model = copy.d` truncated statement, `kernel_pruning_saliency.py`'s stubbed-out `compute_mask` returning `0`, etc.) if a future pass wants to tackle it.

Known follow-up

No runtime execution of either pruning script has happened yet — this machine has no PyTorch installed, and the real dataset/training happens on the `/home3/pragnesh/`-referenced cluster. Before relying on new results, run at least one full round of each script (`channel_pruning_distance.py` , `channel_pruning_saliency.py`) end-to-end on the actual training machine to confirm the `deep_model_copy_channelwise` reslicing logic and BatchNorm handling behave as expected on real VGG16-BN weights.